

Parallel Policy-Gradient Methods for Parameter Optimization of Nonlinear Feedback Controllers

An Nguyen and Leilei Cui, *Member, IEEE*

Abstract—Structured feedback controllers provide rigorous stability guarantees, but their closed-loop performance often depends on manually tuned gains and design parameters. Policy-gradient methods offer a systematic approach to parameter optimization; however, conventional gradient evaluation requires sequential forward state rollout and backward costate propagation. This letter develops a time-parallel policy-gradient framework for discrete-time nonlinear control-affine systems. We derive the policy-gradient expression and recover the standard discrete-time linear-quadratic regulator result as a special case. The state and costate rollouts required for policy-gradient evaluation are formulated as residual-minimization problems and solved using Gauss–Newton (GN) iterations with parallel associative scans. For closed-loop systems that are globally asymptotically stable and locally exponentially stable, we show that the residual-minimization problems satisfy a local Polyak–Łojasiewicz (PL) inequality and that the GN iterates converge locally at a quadratic rate. Moreover, the PL constant, the size of the convergence neighborhood, and the quadratic convergence bound are independent of the rollout horizon T . In addition, we establish global finite-step convergence of the GN state solver: for any finite rollout horizon T , the exact state trajectory is recovered in at most T iterations from any initial guess. Finally, an inertia-wheel pendulum example with interconnection and damping assignment passivity-based control (IDA-PBC) demonstrates improved closed-loop performance and the computational benefits of the proposed parallel policy-gradient framework.

Index Terms—Control-affine systems, nonlinear control, optimal control, policy optimization, parallel computing, policy gradient.

I. INTRODUCTION

NONLINEAR control methods are designed around analytical structure of the system [1], [2], where they can provide stability guarantees while retaining physically meaningful controller structures. Their practical performance, however, often depends strongly on the selection of gains and other design parameters. These parameters are commonly chosen through trial and error or analytical guidelines, which becomes increasingly difficult for nonlinear systems with multiple coupled parameters. Policy-gradient methods offer an

appealing alternative: rather than replacing the controller structure, they can optimize its parameters directly with respect to a performance objective [3], [4]. When optimization is restricted to a range of stabilizing controller parameters, the stability guarantees and physical structure of the original controller can be preserved. The success of policy-gradient methods in reinforcement learning and control further motivates their use as a systematic approach for tuning such structured feedback controllers [5], [6].

For linear-quadratic control, policy gradients admit explicit forms with well-understood convergence and robustness properties [6], [7], whereas nonlinear systems generally require numerical gradient computation [8]. Conventionally, computing the policy gradient requires sequential forward rollout of the state trajectory and backward propagation of the costate over the horizon T . This inherently sequential procedure can become computationally expensive for long-horizon problems and constitutes a major bottleneck of policy-gradient methods. In this paper, we develop a time-parallel approach for evaluating the state and costate trajectories required for policy-gradient computation, enabling substantial acceleration over conventional sequential methods on parallel hardware such as GPUs. Recent work has explored time-parallel evaluation of nonlinear dynamical systems. DEER [9], [10] reformulates sequential trajectory evaluation as an optimization problem involving a residual-based merit function and applies a Gauss–Newton method, with the resulting computations parallelized using associative scans. DeepPCR [11] adopts a related formulation but employs parallel cyclic reduction to exploit temporal parallelism.

The computational efficiency of this reformulation depends on the cost of each optimization step and the number of steps required for convergence. Associative scans provide $\mathcal{O}(\log T)$ parallel computational depth per step [9], [10]. The iteration count also depends on the initialization and the conditioning of the merit function. Existing analyses relate this conditioning and the resulting convergence guarantees to contraction or predictability conditions expressed through bounds on Jacobian products [12]. Global asymptotic stability alone does not ensure such bounds, since trajectories may exhibit noncontractive transients or nonexponential decay.

In this paper, we show that global asymptotic stability together with local exponential stability yields a horizon-independent bound on the inverse residual Jacobian near the exact trajectory, uniformly over a compact set of initial states for fixed controller parameters. The key observation is that global asymptotic stability ensures entry into a sufficiently

This work is supported by the UNM School of Engineering faculty startup funds, the UNM Research Allocations Committee (RAC) grant, and the National Science Foundation under Award No. 2624522.

An Nguyen and Leilei Cui are with the Learning and Control (LC) lab, Department of Mechanical Engineering, University of New Mexico, Albuquerque, NM 87110 USA (email: annguyen2303@unm.edu; lcui@unm.edu).

small neighborhood of the equilibrium after a uniformly bounded number of steps, while local exponential stability provides contraction of the Jacobians there in an appropriate norm. Consequently, only a uniformly bounded initial portion of the trajectory may be noncontractive. Smoothness ensures that the resulting Jacobian-product bounds also hold for nearby trajectory estimates. This establishes a local Polyak-Łojasiewicz (PL) inequality and local linear and quadratic convergence of the state solver, with constants and a neighborhood radius independent of the horizon.

The main contributions of this letter are:

- 1) We derive a policy-gradient formula for discrete-time control-affine nonlinear systems that is consistent with the deterministic policy gradient in [13] and recovers the standard LQR policy gradient as a special case.
- 2) We develop a time-parallel method for optimizing nonlinear feedback-controller parameters by evaluating the state and costate trajectories using Gauss-Newton updates and associative scans that are amenable to GPU parallelization.
- 3) We establish local inverse-Jacobian and PL bounds under closed-loop global asymptotic stability and local exponential stability, yielding local linear and quadratic convergence with horizon-independent constants and basin radius. Separately, the causal residual structure ensures that the exact full-step state solver terminates within T iterations for any finite horizon T and arbitrary initialization, while the fixed-state costate problem is solved in one exact step.

The proposed approach is evaluated on discrete-time LQR and the inertia-wheel pendulum.

II. POLICY OPTIMIZATION FOR NONLINEAR SYSTEMS

In this paper, we consider the discrete-time nonlinear system

$$x_{k+1} = f(x_k) + g(x_k)u_k, \quad k = 0, 1, 2, \dots$$

where $x_k \in \mathbb{R}^n$ denotes the state, $u_k \in \mathbb{R}^m$ denotes the control input, and $f : \mathbb{R}^n \rightarrow \mathbb{R}^n$ and $g : \mathbb{R}^n \rightarrow \mathbb{R}^{n \times m}$ are assumed to be smooth, with $f(0) = 0$. The feedback policy is parameterized as $u_k = \pi(x_k, \theta)$, where $\pi : \mathbb{R}^n \times \mathbb{R}^p \rightarrow \mathbb{R}^m$ is a smooth function and $\theta \in \mathbb{R}^p$ denotes the controller parameter vector.

Define the discrete-time trajectory as $x(k, x_0, \theta)$, which is generated by the closed-loop dynamics starting from x_0 under the policy $u_k = \pi(x_k, \theta)$. The cost functional is defined as

$$\mathcal{J}(x_0, \theta) = \sum_{k=0}^{\infty} l(x(k, x_0, \theta), \theta),$$

where

$$l(x, \theta) = q(x) + \pi(x, \theta)^\top R \pi(x, \theta),$$

$q : \mathbb{R}^n \rightarrow \mathbb{R}_+$ is a positive-definite state-cost function and $R \in \mathbb{S}_{++}^m$ is the control cost matrix. For all k , the closed-loop dynamics is given by

$$x_{k+1} := F(x_k, \theta) = f(x_k) + g(x_k)\pi(x_k, \theta). \quad (1)$$

Definition 2.1: A policy $\pi(x_k, \theta)$ is admissible if the closed-loop system (1) is globally asymptotically stable (GAS) and locally exponentially stable (LES) at the origin.

We aim to develop a gradient-based method for solving

$$\min_{\theta \in \Theta} \bar{\mathcal{J}}(\theta), \quad \bar{\mathcal{J}}(\theta) := \mathbb{E}_{x_0 \sim \mathcal{X}} [\mathcal{J}(x_0, \theta)],$$

where Θ denotes the set of admissible policy parameters associated with controllers designed using classical nonlinear control methods, and $\mathcal{X}$ is a distribution for initial states supported on a compact set Ω . The policy gradient $\nabla_{\theta} \mathcal{J}(x_0, \theta)$ is derived in the following theorem.

Theorem 2.1: Let $\pi(x, \theta)$ be an admissible policy. Consider the following discrete-time Hamiltonian:

$$\mathcal{H}(x_k, \lambda_{k+1}, \theta) = l(x_k, \theta) + \lambda_{k+1}^\top F(x_k, \theta),$$

with the discrete-time Hamiltonian dynamics given by:

$$x_{k+1} = \frac{\partial \mathcal{H}(x_k, \lambda_{k+1}, \theta)}{\partial \lambda_{k+1}}, \quad x_0 \in \Omega, \quad (2a)$$

$$\lambda_k = \frac{\partial \mathcal{H}(x_k, \lambda_{k+1}, \theta)}{\partial x_k}, \quad \lim_{k \rightarrow \infty} \lambda_k = 0. \quad (2b)$$

The policy gradient of the cost functional $\mathcal{J}(x_0, \theta)$ is

$$\begin{aligned} \nabla_{\theta} \mathcal{J}(x_0, \theta) &= \sum_{k=0}^{\infty} (\nabla_{\theta} \pi(x_k, \theta))^\top \left(2R \pi(x_k, \theta) + g(x_k)^\top \lambda_{k+1} \right). \end{aligned} \quad (3)$$

Proof: Define the sensitivity function as

$$S(k, x_0, \theta) = \frac{\partial x(k, x_0, \theta)}{\partial \theta} \in \mathbb{R}^{n \times p}.$$

Taking the derivative of the closed-loop dynamics with respect to θ gives

$$S(k+1, x_0, \theta) = \nabla_x F(x_k, \theta) S(k, x_0, \theta) + \nabla_{\theta} F(x_k, \theta). \quad (4)$$

The costate dynamics in (2) can be rewritten as

$$\lambda_k := G(x_k, \lambda_{k+1}, \theta) = \nabla_x l(x_k, \theta) + \nabla_x F(x_k, \theta)^\top \lambda_{k+1}. \quad (5)$$

Hence, by (4) and (5) we can compute

$$\begin{aligned} S(k+1, x_0, \theta)^\top \lambda_{k+1} &= \nabla_{\theta} F(x_k, \theta)^\top \lambda_{k+1} + S(k, x_0, \theta)^\top \lambda_k \\ &\quad - S(k, x_0, \theta)^\top \nabla_x l(x_k, \theta). \end{aligned}$$

Group the terms and define the forward difference:

$$\Delta(S(k, x_0, \theta)^\top \lambda_k) = S(k+1, x_0, \theta)^\top \lambda_{k+1} - S(k, x_0, \theta)^\top \lambda_k.$$

Therefore, it follows that

$$\begin{aligned} \Delta(S(k, x_0, \theta)^\top \lambda_k) &= -S(k, x_0, \theta)^\top \nabla_x l(x_k, \theta) + \nabla_{\theta} F(x_k, \theta)^\top \lambda_{k+1}. \end{aligned}$$

Additionally, the derivative of the cost functional with respect to θ , is:

$$\nabla_{\theta} \mathcal{J}(x_0, \theta) = \sum_{k=0}^{\infty} \left(S(k, x_0, \theta)^\top \nabla_x l(x_k, \theta) + \nabla_{\theta} l(x_k, \theta) \right).$$

Substituting the expression of $\Delta(S(k, x_0, \theta)^\top \lambda_k)$ into the above equation gives

$$\begin{aligned} \nabla_\theta \mathcal{J}(x_0, \theta) &= \sum_{k=0}^{\infty} \left(\nabla_\theta F(x_k, \theta)^\top \lambda_{k+1} + \nabla_\theta l(x_k, \theta) \right) \\ &\quad - \lim_{k \rightarrow \infty} (S(k, x_0, \theta)^\top \lambda_k) + S(0, x_0, \theta)^\top \lambda_0. \end{aligned}$$

In the above equation, since $\lim_{k \rightarrow \infty} \lambda_k = 0$ and $S(0, x_0, \theta) = 0$, the boundary terms vanish. Hence, we have

$$\nabla_\theta \mathcal{J}(x_0, \theta) = \sum_{k=0}^{\infty} \left(\nabla_\theta F(x_k, \theta)^\top \lambda_{k+1} + \nabla_\theta l(x_k, \theta) \right).$$

It follows from the closed-loop dynamic and the cost function that

$$\begin{aligned} \nabla_\theta F(x_k, \theta)^\top &= \nabla_\theta \pi(x_k, \theta)^\top g(x_k)^\top, \\ \nabla_\theta l(x_k, \theta) &= 2 \nabla_\theta \pi(x_k, \theta)^\top R \pi(x_k, \theta), \end{aligned}$$

which, in turn, results in (3). $\blacksquare$

Remark 2.1: By the Bellman equation, the cost function satisfies

$$\mathcal{J}(x_k, \theta) = l(x_k, \theta) + \mathcal{J}(F(x_k, \theta), \theta).$$

Differentiating both sides with respect to x_k gives

$$\nabla_x \mathcal{J}(x_k, \theta) = \nabla_x l(x_k, \theta) + \nabla_x F(x_k, \theta)^\top \nabla_x \mathcal{J}(x_{k+1}, \theta).$$

Therefore, by comparing with (5), the costate admits the interpretation

$$\lambda_k = \nabla_x \mathcal{J}(x_k, \theta).$$

Remark 2.2: Define the state occupation measure induced by the policy $\pi(\cdot, \theta)$ and the initial-state distribution $\mathcal{X}$ as

$$\rho_\theta(dx) := \mathbb{E}_{x_0 \sim \mathcal{X}} \left[\sum_{k=0}^{\infty} \delta_{x(k, x_0, \theta)}(dx) \right].$$

Then, for any integrable function ϕ ,

$$\int_{\mathbb{R}^n} \phi(x) \rho_\theta(dx) = \mathbb{E}_{x_0 \sim \mathcal{X}} \left[\sum_{k=0}^{\infty} \phi(x(k, x_0, \theta)) \right].$$

Using the Q -function

$$Q(x, u, \theta) := q(x) + u^\top R u + \mathcal{J}(f(x) + g(x)u, \theta),$$

we have

$$\nabla_u Q(x, u, \theta) = 2Ru + g(x)^\top \nabla_x \mathcal{J}(f(x) + g(x)u, \theta).$$

Therefore, by Remark 2.1, the policy gradient in Theorem 2.1 can be equivalently written as

$$\nabla_\theta \bar{\mathcal{J}}(\theta) = \int_{\mathbb{R}^n} (\nabla_\theta \pi(x, \theta))^\top \nabla_u Q(x, u, \theta)|_{u=\pi(x, \theta)} \rho_\theta(dx).$$

This expression has the same form as the deterministic policy gradient theorem in [13], with ρ_θ playing the role of the state visitation measure.

For linear time-invariant (LTI) systems with quadratic costs, the policy-gradient expression in Theorem 2.1 reduces to the standard LQR policy gradient derived in [6].

Corollary 2.1: Consider the LTI system with $f(x_k) = Ax_k$ and $g(x_k) = B$, where $A \in \mathbb{R}^{n \times n}$ and $B \in \mathbb{R}^{n \times m}$, together with the quadratic state cost $q(x_k) = x_k^\top Q x_k$, where $Q \in \mathbb{S}_{++}^n$. Then, for any stabilizing linear policy $u_k = -Kx_k$, the policy gradient is given by

$$\nabla_K \mathcal{J}(x_0, K) = 2[(R + B^\top P_K B)K - B^\top P_K A] \Sigma_K,$$

where $\Sigma_K := \sum_{k=0}^{\infty} x_k x_k^\top$, and $P_K = P_K^\top \succ 0$ is the solution to

$$P_K = (A - BK)^\top P_K (A - BK) + Q + K^\top R K.$$

Proof: It follows from Theorem 2.1 that, for the LQR problem,

$$\nabla_K \mathcal{J}(x_0, K) = \sum_{k=0}^{\infty} (2RKx_k - B^\top \lambda_{k+1}) x_k^\top. \quad (6)$$

Since K is stabilizing, the corresponding value function is quadratic [14], i.e. $\mathcal{J}(x_k, K) = x_k^\top P_K x_k$. Hence, by Remark 2.1,

$$\lambda_{k+1} = \nabla_{x_{k+1}} \mathcal{J}(x_{k+1}, K) = 2P_K x_{k+1} = 2P_K (A - BK)x_k.$$

Substituting this expression into (6) yields

$$\begin{aligned} \nabla_K \mathcal{J}(x_0, K) &= 2[RK - B^\top P_K (A - BK)] \sum_{k=0}^{\infty} x_k x_k^\top \\ &= 2[(R + B^\top P_K B)K - B^\top P_K A] \Sigma_K, \end{aligned}$$

which completes the proof. $\blacksquare$

III. PARALLELIZED MONTE CARLO APPROXIMATION OF THE POLICY GRADIENT

In this section, we employ a Monte Carlo method to approximate $\nabla_\theta \bar{\mathcal{J}}(\theta) = \mathbb{E}_{x_0 \sim \mathcal{X}} [\nabla_\theta \mathcal{J}(x_0, \theta)]$. Specifically, we draw N initial states $\{x_{0,j}\}_{j=1}^N$ independently and identically distributed (i.i.d.) from $\mathcal{X}$ with j is the index of the j -th sample, and approximate the policy gradient by

$$\hat{\nabla}_\theta \bar{\mathcal{J}}(\theta) = \frac{1}{N} \sum_{j=1}^N \nabla_\theta \mathcal{J}(x_{0,j}, \theta). \quad (7)$$

Let $x(k, x_{0,j}, \theta)$ and $\lambda(k, x_{0,j}, \theta)$ denote the state and costate trajectories of (2), respectively, with $x_0 = x_{0,j}$ and $\lim_{k \rightarrow \infty} \lambda(k, x_{0,j}, \theta) = 0$. Since $\theta \in \Theta$ is an admissible parameter vector, the corresponding closed-loop trajectories satisfy $\lim_{k \rightarrow \infty} x(k, x_{0,j}, \theta) = 0$ and $\lim_{k \rightarrow \infty} \lambda(k, x_{0,j}, \theta) = 0$. Therefore, the tail contribution in the policy-gradient expression of Theorem 2.1 vanishes asymptotically, and the infinite-horizon policy gradient can be approximated using state and costate trajectories over a sufficiently large finite horizon T .

It follows from Theorem 2.1 that computing $\nabla_\theta \mathcal{J}(x_{0,j}, \theta)$ requires evaluating the Hamiltonian dynamics (2) over $k = 0, \dots, T$. These recursions are inherently sequential and can become computationally expensive for long horizons. To alleviate this temporal bottleneck, we reformulate the sequential state and costate recursions as a root-finding optimization problem and exploit associative scans [15] to parallelize the trajectory evaluation across time.

Consider the stacked state and costate estimates

$$s_{x,j} = [\xi_1^\top, \dots, \xi_T^\top]^\top \in \mathbb{R}^{nT},$$

$$s_{\lambda,j} = [\phi_{T-1}^\top, \dots, \phi_0^\top]^\top \in \mathbb{R}^{nT},$$

which approximate the corresponding state and costate trajectories

$$s_{x,j}^* = [x(1, x_{0,j}, \theta)^\top, \dots, x(T, x_{0,j}, \theta)^\top]^\top \in \mathbb{R}^{nT},$$

$$s_{\lambda,j}^* = [\lambda(T-1, x_{0,j}, \theta)^\top, \dots, \lambda(0, x_{0,j}, \theta)^\top]^\top \in \mathbb{R}^{nT},$$

respectively. Hereafter, the subscript j , and the policy parameter θ are omitted for notational simplicity whenever no confusion arises. In particular, we write $x_k = x(k, x_{0,j}, \theta)$ and $\lambda_k = \lambda(k, x_{0,j}, \theta)$. The residuals associated with the stacked state and costate estimates are then defined as

$$r_x(s_x) = [\xi_1^\top - F(\xi_0)^\top, \dots, \xi_T^\top - F(\xi_{T-1})^\top]^\top,$$

$$r_\lambda(s_\lambda) = [\phi_{T-1}^\top - G(x_{T-1}, \phi_T)^\top, \dots, \phi_0^\top - G(x_0, \phi_1)^\top]^\top.$$

The two trajectories s_x^* and s_λ^* are evaluated using two separate Gauss-Newton passes. The first pass evaluates s_x^* from a given initial state $\xi_0 = x_{0,j}$. The resulting state trajectory is then held fixed, the terminal condition $\phi_T = \nabla_x l(x_T)$ is evaluated, and the second pass evaluates s_λ^* backward in time through (5). In each pass, we solve the following residual-minimization problem for both the state and costate:

$$\min_{s_x \in \mathbb{R}^{nT}} \mathcal{L}_x(s_x) = \frac{1}{2} \|r_x(s_x)\|_2^2 \quad (8a)$$

$$\min_{s_\lambda \in \mathbb{R}^{nT}} \mathcal{L}_\lambda(s_\lambda) = \frac{1}{2} \|r_\lambda(s_\lambda)\|_2^2 \quad (8b)$$

The residual Jacobian $J(s_x) = \partial r_x(s_x)/\partial s_x$ and $J(s_\lambda) = \partial r_\lambda(s_\lambda)/\partial s_\lambda$ are square and nonsingular. Consequently, the Gauss-Newton step for (8) is

$$s_x^{(i+1)} - s_x^{(i)} = \Delta s_x^{(i)} = -J(s_x^{(i)})^{-1} r_x(s_x^{(i)}), \quad (9a)$$

$$s_\lambda^{(i+1)} - s_\lambda^{(i)} = \Delta s_\lambda^{(i)} = -J(s_\lambda^{(i)})^{-1} r_\lambda(s_\lambda^{(i)}). \quad (9b)$$

The Jacobian $J(s_x)$ has a block diagonal structure

$$J(s_x) = \begin{bmatrix} I_n & 0 & \cdots & 0 & 0 \\ -\nabla_x F(\xi_0) & I_n & \cdots & 0 & 0 \\ \vdots & \vdots & \ddots & \vdots & \vdots \\ 0 & 0 & \cdots & I_n & 0 \\ 0 & 0 & \cdots & -\nabla_x F(\xi_{T-1}) & I_n \end{bmatrix}.$$

Similarly, $J(s_\lambda)$ is obtained by replacing $\nabla_x F(\xi_k)$ in the above expression with $\nabla_\lambda G(x_{T-k-1}, \phi_{T-k})$. Since $\Delta \xi_0^{(i)} = 0$, the state correction satisfies

$$\Delta \xi_{k+1}^{(i)} = \nabla_x F(\xi_k^{(i)}, \theta) \Delta \xi_k^{(i)} - r_{x,k+1}^{(i)}, \quad (10)$$

where $\Delta \xi_{k+1}^{(i)} = \xi_{k+1}^{(i+1)} - \xi_{k+1}^{(i)}$. During the costate pass, the state trajectory and terminal costate are held fixed. Hence, $\Delta \phi_T^{(i)} = 0$, $\Delta \phi_k^{(i)} = \phi_k^{(i+1)} - \phi_k^{(i)}$, and the costate correction satisfies

$$\Delta \phi_k^{(i)} = \nabla_\lambda G(x_k, \phi_{k+1}^{(i)}, \theta) \Delta \phi_{k+1}^{(i)} - r_{\lambda,k}^{(i)}. \quad (11)$$

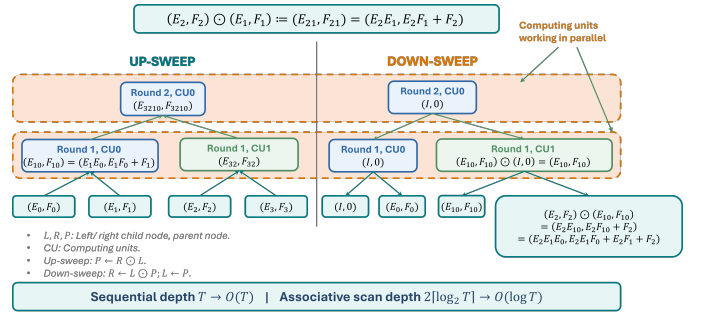


Fig. 1. Associative-scan evaluation example of the affine recursion for $T = 4$.

A. Parallel evaluation by associative scan

Since the state and costate correction equations are affine, one can parallelize the root-finding problem using an associative scan [10]. An illustration of associative scan is given in Figure 1. Consider an affine recursion of the form $z_{k+1} = E_k z_k + F_k$. Each update can be viewed as an affine map $z \mapsto E_k z + F_k$, represented by the pair (E_k, F_k) . The composition of two affine maps is represented by the binary operator

$$(E_2, F_2) \odot (E_1, F_1) := (E_2 E_1, E_2 F_1 + F_2).$$

This operator is associative because it corresponds directly to function composition. Consequently, the sequence of affine recursions can be evaluated through a parallel prefix scan, which computes all cumulative affine-map compositions simultaneously.

For the state pass, define $\mathbf{a}_{x_k}^{(i)} := (\nabla_x F(\xi_k^{(i)}, \theta), -r_{x,k+1}^{(i)})$. Then all state corrections are recovered from the prefix scan

$$\Delta \xi_{k+1}^{(i)} = \left(\mathbf{a}_{x_k}^{(i)} \odot \cdots \odot \mathbf{a}_{x_0}^{(i)} \right) [\Delta \xi_0^{(i)}].$$

Similarly, for the fixed-state costate pass, define $\mathbf{a}_{\phi_k}^{(i)} := (\nabla_\lambda G(x_k, \phi_{k+1}^{(i)}, \theta), -r_{\lambda,k}^{(i)})$, the costate corrections are obtained from the suffix scan

$$\Delta \phi_k^{(i)} = \left(\mathbf{a}_{\phi_k}^{(i)} \odot \cdots \odot \mathbf{a}_{\phi_{T-1}}^{(i)} \right) [\Delta \phi_T^{(i)}].$$

At each Gauss-Newton iteration, residuals and Jacobian blocks are evaluated in parallel across time and combined by an associative scan, yielding $\mathcal{O}(\log T)$ parallel depth and $\mathcal{O}(T)$ total compositions.

Algorithm 1 estimates $\nabla_\theta \bar{\mathcal{J}} = \mathbb{E}_{x_0} [\nabla_\theta \mathcal{J}(x_0, \theta)]$ using Monte Carlo samples. For each initial condition, a forward Gauss-Newton pass computes the state trajectory and a reverse pass computes the costate trajectory. The resulting gradients are averaged to update θ , while the projected gradient descent keeps θ in the admissible region Θ .

B. Convergence Analysis of the Gauss-Newton Iterations

Fix $\theta \in \Theta$ and omit it from F for simplicity. GAS and compactness of Ω imply uniform boundedness of the exact

Algorithm 1 Monte Carlo Policy-Gradient Optimization

Require: Initial policy parameter $\theta, \theta_{\text{old}} \in \Theta$, horizon T , number of Monte Carlo samples N , initial-state distribution $\mathcal{X}$, policy step sizes $\{\eta_\ell\}$, tolerance ϵ_D , and policy-gradient tolerance ϵ_{PG}

Ensure: Optimized policy parameter θ

```

1:  $\ell \leftarrow 0$ 
2: repeat
3:   Sample  $N$  initial states  $\{x_{0,j}\}_{j=1}^N \stackrel{\text{i.i.d.}}{\sim} \mathcal{X}$ 
4:   for all  $j = 1, \dots, N$  in parallel do
5:     Initialize  $s_{x,j} \in \mathbb{R}^{nT}$ 
6:     repeat
7:       Compute  $\Delta s_{x,j}$  from (10) by associative scan
8:        $s_{x,j} \leftarrow s_{x,j} + \Delta s_{x,j}$ 
9:     until  $\|\Delta s_{x,j}\| \leq \epsilon_D$ 
10:    Initialize  $s_{\lambda,j} \in \mathbb{R}^{nT}$ 
11:    repeat
12:      Compute  $\Delta s_{\lambda,j}$  from (11) by associative scan
13:       $s_{\lambda,j} \leftarrow s_{\lambda,j} + \Delta s_{\lambda,j}$ 
14:    until  $\|\Delta s_{\lambda,j}\| \leq \epsilon_D$ 
15:    Compute  $\nabla_\theta \mathcal{J}(x_{0,j}, \theta)$  from (3),  $s_{x,j}$  and  $s_{\lambda,j}$ 
16:  end for
17:  Compute  $\hat{\nabla}_\theta \bar{\mathcal{J}}(\theta)$  from (7)
18:   $\theta_{\text{old}} \leftarrow \theta$ 
19:   $\theta \leftarrow \text{Proj}_\Theta \left( \theta_{\text{old}} - \eta_\ell \hat{\nabla}_\theta \bar{\mathcal{J}}(\theta_{\text{old}}) \right)$ 
20:   $\ell \leftarrow \ell + 1$ 
21: until  $\|\theta_{\text{old}} - \theta\| \leq \epsilon_{PG}$ 
22: return  $\theta$ 

```

trajectories. Let K be a compact convex neighborhood of their closure. Define

$$\|v\|_\infty := \max_k \|v_k\|_2, \quad e^{(i)} := s_x^{(i)} - s_x^*.$$

Lemma 3.1: Let $L > 0$ be a Lipschitz bound for $\nabla_x F$ on K , which exists by smoothness. Then

$$\|J(s_1) - J(s_2)\|_2 \leq L \|s_1 - s_2\|_\infty, \quad s_1, s_2 \in K^T.$$

Proof: For $s_1 = (z_1, \dots, z_T)$ and $s_2 = (y_1, \dots, y_T)$, the Jacobian difference has only the subdiagonal blocks $\nabla_x F(y_k) - \nabla_x F(z_k)$. Hence, for $T \geq 2$,

$$\begin{aligned} \|J(s_1) - J(s_2)\|_2 &= \max_{1 \leq k < T} \|\nabla_x F(z_k) - \nabla_x F(y_k)\|_2 \\ &\leq L \|s_1 - s_2\|_\infty. \end{aligned}$$

Lemma 3.2: There exist $\eta_\theta, M_\theta > 0$, independent of T , such that,

$$\|J(s_x)^{-1}\|_2 \leq M_\theta. \quad (12)$$

for any $s_x \in \mathbb{R}^{nT}$ satisfying $\|s_x - s_x^*\|_\infty \leq \eta_\theta$,

Proof: Smoothness and LES imply that $\nabla_x F(0)$ is Schur stable. The discrete Lyapunov inequality and continuity give $O \succ 0$, $c \in (0, 1)$, and $\eta_\theta > 0$ such that

$$\|O \nabla_x F(z) O^{-1}\|_2 \leq c \quad \text{for } \|z\|_2 \leq 2\eta_\theta.$$

Shrink η_θ so that the corresponding neighborhoods of all exact trajectories lie in K . GAS and compactness of Ω give a uniform finite τ such that $\|x_k^*\|_2 \leq \eta_\theta$ for all $k \geq \tau$.

For $\|s_x - s_x^*\|_\infty \leq \eta_\theta$, all ξ_k lie in K , and $\|\xi_k\|_2 \leq 2\eta_\theta$ for $k \geq \tau$. Thus, each consecutive Jacobian product has at most τ factors outside the contracting neighborhood. Since K is compact, define

$$\alpha := \max \left\{ 1, \max_{z \in K} \|O \nabla_x F(z) O^{-1}\|_2 \right\},$$

$$C_\theta := \|O^{-1}\|_2 \|O\|_2 \left(\frac{\alpha}{c} \right)^\tau.$$

Each product of ℓ consecutive Jacobians contains at most $\min\{\ell, \tau\}$ factors outside the contracting neighborhood. Since $\alpha \geq 1 > c$,

$$\begin{aligned} \|\nabla_x F(\xi_{j+\ell-1}) \cdots \nabla_x F(\xi_j)\|_2 &\leq \|O^{-1}\|_2 \|O\|_2 c^\ell \left(\frac{\alpha}{c} \right)^{\min\{\ell, \tau\}} \\ &\leq C_\theta c^\ell. \end{aligned} \quad (13)$$

Write $J = I - N$. Since $N^T = 0$,

$$J^{-1} = \sum_{\ell=0}^{T-1} N^\ell.$$

Each N^ℓ is a block shift whose nonzero blocks are the products in (13). Consequently,

$$\|N^\ell\|_2 \leq C_\theta c^\ell, \quad \|N^\ell v\|_\infty \leq C_\theta c^\ell \|v\|_\infty.$$

Summing these gives (12) with $M_\theta = C_\theta / (1 - c)$. ■

As a direct consequence of Lemma 3.2, the following result establishes that the merit function $\mathcal{L}_x$ satisfies a local PL condition.

Corollary 3.1: For $\|s_x - s_x^*\|_\infty \leq \eta_\theta$, the merit function satisfies

$$\frac{1}{2} \|\nabla \mathcal{L}_x(s_x)\|_2^2 \geq M_\theta^{-2} \mathcal{L}_x(s_x). \quad (14)$$

Proof: Lemma 3.2 gives $\sigma_{\min}(J(s_x)) \geq M_\theta^{-1}$. Since $\nabla \mathcal{L}_x(s_x) = J(s_x)^\top r_x(s_x)$,

$$\frac{1}{2} \|\nabla \mathcal{L}_x(s_x)\|_2^2 \geq \frac{1}{2M_\theta^2} \|r_x(s_x)\|_2^2 = M_\theta^{-2} \mathcal{L}_x(s_x). \quad \blacksquare$$

Theorem 3.1: Define

$$\delta_\theta := \min \left\{ \eta_\theta, \frac{1}{LM_\theta} \right\}. \quad (15)$$

If $\|e^{(0)}\|_\infty \leq \delta_\theta$, the exact full-step Gauss-Newton iterates remain in this neighborhood and satisfy

$$\|e^{(i+1)}\|_2 \leq \frac{1}{2} \|e^{(i)}\|_2. \quad (16)$$

Proof: Suppose $\|e^{(i)}\|_\infty \leq \delta_\theta$. Applying the fundamental theorem of calculus along the segment from s_x^* to $s_x^{(i)}$ gives

$$r_x(s_x^{(i)}) = \int_0^1 J(s_x^* + te^{(i)}) e^{(i)} dt,$$

where $r_x(s_x^*) = 0$ was used. Subtracting s_x^* from the Gauss-Newton update and substituting this identity yields

$$\begin{aligned} e^{(i+1)} &= e^{(i)} - J(s_x^{(i)})^{-1} r_x(s_x^{(i)}) \\ &= J(s_x^{(i)})^{-1} \int_0^1 \left[J(s_x^{(i)}) - J(s_x^* + te^{(i)}) \right] e^{(i)} dt. \end{aligned}$$

By Lemma 3.1,

$$\|J(s_x^{(i)}) - J(s_x^* + te^{(i)})\|_2 \leq L(1-t)\|e^{(i)}\|_\infty.$$

Applying this estimate, its blockwise counterpart, and Lemma 3.2 gives

$$\|e^{(i+1)}\|_\infty \leq \frac{LM_\theta}{2} \|e^{(i)}\|_\infty^2, \quad (17)$$

$$\|e^{(i+1)}\|_2 \leq \frac{LM_\theta}{2} \|e^{(i)}\|_\infty \|e^{(i)}\|_2, \quad (18)$$

where $\int_0^1 (1-t) dt = 1/2$. Since $LM_\theta \delta_\theta \leq 1$, the first inequality preserves the neighborhood and the second proves (16). Induction completes the proof. ■

Corollary 3.2: The neighborhood $\|s_x - s_x^*\|_\infty \leq \delta_\theta$, with δ_θ given by (15), is a horizon-independent basin of quadratic convergence. For any initialization in this neighborhood,

$$\|e^{(i+1)}\|_2 \leq \frac{LM_\theta}{2} \|e^{(i)}\|_2^2,$$

$$\|r_x(s_x^{(i+1)})\|_2 \leq \frac{LM_\theta^2}{2} \|r_x(s_x^{(i)})\|_2^2.$$

Proof: The error bound follows from (18) and $\|e\|_\infty \leq \|e\|_2$. For $h = -J(s)^{-1}r_x(s)$, we have $r_x(s) + J(s)h = 0$. Both iterations and their connecting segment remain in the neighborhood by Theorem 3.1. Taylor's formula and Lemma 3.1 give

$$\|r_x(s+h)\|_2 \leq \frac{L}{2} \|h\|_2^2 \leq \frac{LM_\theta^2}{2} \|r_x(s)\|_2^2.$$

Theorem 3.2: For any finite T and arbitrary initialization, the state iteration in (9a) terminates in at most T steps. Moreover, with the states and terminal costate fixed, the costate iteration in (9b) terminates in one exact step.

Proof: The state update satisfies

$$\xi_{k+1}^{(i+1)} = F(\xi_k^{(i)}) + \nabla_x F(\xi_k^{(i)})(\xi_k^{(i+1)} - \xi_k^{(i)}).$$

Since $\xi_0^{(i)} = x_0$, induction gives $\xi_k^{(i)} = x_k$ for $k \leq \min\{i, T\}$. Thus $s_x^{(i)} = s_x^*$ for all $i \geq T$.

The fixed-state costate residual is affine with a nonsingular Jacobian, so one exact step solves it. ■

IV. NUMERICAL EXPERIMENTS

All experiments are run on a workstation equipped with an Intel Core Ultra 9 285K CPU, 128 GB of RAM, and an NVIDIA GeForce RTX 5090 GPU with 32 GB of memory. We use JAX [16] throughout the experiments and use the implementation of DEER given in [12] for the Gauss-Newton method. All details for reproducing the experiments are available at <https://github.com/lc-lab25/Parallel-Policy-Gradient.git>.

A. Discrete-time LQR

In this section, the LQR experiment is used to validate the policy-gradient expression and evaluate the computational scaling of the parallel trajectory solver. We consider a randomly generated system matrix A and choose an initial feedback controller $u = -Kx$ such that the resulting closed-loop

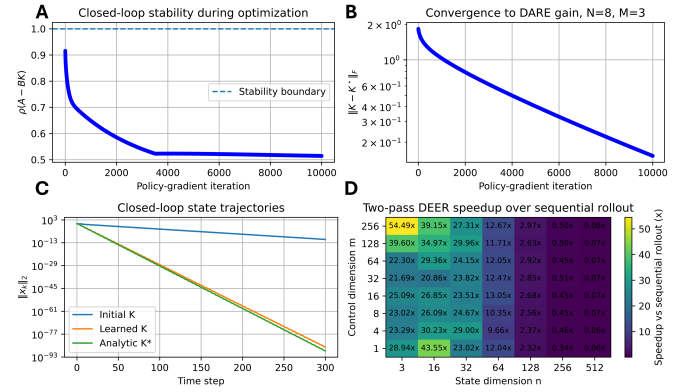


Fig. 2. LQR validation and computational performance. A: Stability of LQR system under the controller during optimization. B: Difference between the optimized gain and analytic optimal gain through time. C: Trajectory errors through time with the initial, learned, and analytical gain. D: Performance speedup when using the method comparing to sequential roll out.

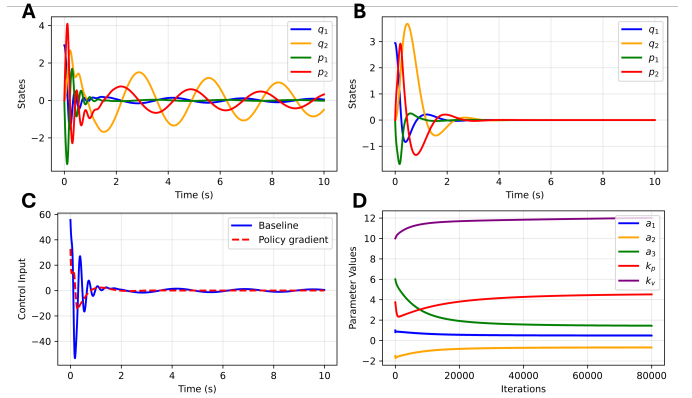


Fig. 3. Inertia wheel pendulum experiments. A: Baseline closed-loop state of the inertia-wheel pendulum. B: Optimized closed-loop state of the inertia-wheel pendulum. C: Comparing the baseline and policy-gradient controllers. D: Controller-parameter during policy-gradient optimization.

system is stable. The stage cost is taken as $l = x^\top x + u^\top u$, and the controller is updated according to the policy-gradient expression in (6). The stability of the controller at each iteration of Algorithm 1 is verified and shown in Figure 2A. Figure 2B shows the convergence of the policy-gradient gain toward the analytical optimal LQR gain. Figure 2C reports the state-trajectory errors. Finally, Figure 2D compares the computational time of the parallel trajectory solver with conventional sequential rollout for sampled LQR trajectories on a CPU-based workstation. Each reported result is averaged over 20 independent runs.

B. Inertia Wheel Pendulum

1) System Description and IDA-PBC Controller Design:

We evaluate the proposed method on the nonlinear underactuated inertia-wheel pendulum. Since the open-loop system is unstable, we first restrict the policy to a stabilizing IDA-PBC class

and then optimize its parameters using the proposed parallel policy gradient.

The dynamics of the inertia-wheel pendulum are given in [17] as

$$\begin{bmatrix} I_1 + I_2 & I_2 \\ I_2 & I_2 \end{bmatrix} \begin{bmatrix} \ddot{\omega}_1 \\ \ddot{\omega}_2 \end{bmatrix} + \begin{bmatrix} -mgL \sin \omega_1 \\ 0 \end{bmatrix} = \begin{bmatrix} 0 \\ 1 \end{bmatrix} u. \quad (19)$$

Here, ω_1 denotes the pendulum angle, ω_2 denotes the disk angle relative to the pendulum, and u denotes the motor torque. As shown in [17], a stabilizing controller can be constructed in the form

$$\begin{aligned} u &= \pi(x, \theta) \\ &= \gamma_1 \sin(q_1) + k_p(q_2 + \gamma_2 q_1) + k_v k_2(\dot{q}_2 + \gamma_2 \dot{q}_1). \end{aligned} \quad (20)$$

Here, $[q_1, q_2]^\top = [\omega_1, \omega_1 + \omega_2]^\top$ and k_p and k_v are the proportional and damping-injection gains, respectively. The constants γ_1 , γ_2 and k_2 are determined by the components of the desired inertia matrix

$$\begin{aligned} M_d &= \begin{bmatrix} a_1 & a_2 \\ a_2 & a_3 \end{bmatrix}, & k_2 &= -I \frac{a_1 + a_2}{a_1 a_3 - a_2^2}, \\ \gamma_1 &= \frac{a_2}{a_1 + a_2} mgL, & \gamma_2 &= -\frac{I_1 a_2 + a_3}{I_2 a_1 + a_2}. \end{aligned}$$

The controller parameters $\theta = [k_p, k_v, a_1, a_2, a_3]^\top$ are required to satisfy the following constraints to ensure asymptotic stability:

$$\Theta := \left\{ \theta \in \mathbb{R}^5 \mid \begin{array}{l} k_p > 0, \quad k_v > 0, \quad a_1 > 0, \\ a_1 a_3 - a_2^2 > 0, \quad -(a_1 + a_2) > 0 \end{array} \right\}. \quad (21)$$

For any $\theta \in \Theta$, the origin of the closed-loop system is asymptotically stable, with a domain of attraction equal to the entire state space except for a set of Lebesgue measure zero. The following proposition further establishes that the closed-loop system is locally exponentially stable under the controller $\pi(x, \theta)$ for any $\theta \in \Theta$. Therefore, Θ constitutes the admissible set of controller parameters.

Proposition 4.1: The inertia wheel system (19) under the controller given in (20) is locally exponentially stable at the origin.

Proof: Linearizing (19) and (20) at the origin gives

$$\nabla_x F(0, \theta) = \begin{bmatrix} 0 & 0 & \frac{1}{I_1} & 0 \\ 0 & 0 & 0 & \frac{1}{I_2} \\ mgL - \gamma_1 - k_p \gamma_2 & -k_p & -\frac{k_v k_2 \gamma_2}{I_1} & -\frac{k_v k_2}{I_2} \\ \gamma_1 + k_p \gamma_2 & k_p & \frac{k_v k_2 \gamma_2}{I_1} & \frac{k_v k_2}{I_2} \end{bmatrix}.$$

Also, $D > 0$, $S < 0$, and $E = \frac{S^2 + D}{a_1} > 0$, where

$$D = a_1 a_3 - a_2^2, \quad S = a_1 + a_2, \quad E = a_1 + 2a_2 + a_3.$$

Using the expressions for γ_1, γ_2 , and k_2 , direct expansion gives $\det(\lambda I - A) = b_4 \lambda^4 + b_3 \lambda^3 + b_2 \lambda^2 + b_1 \lambda + b_0$, where

$$\begin{cases} b_4 = I_1 I_2, \\ b_3 = \frac{k_v I_1 I_2 E}{D}, \\ b_2 = -\frac{I_2 mgL a_1 + k_p I_1 E}{S}, \\ b_1 = -\frac{k_v I_2 S mgL}{D}, b_0 = k_p mgL. \end{cases}$$

All coefficients are strictly positive. Moreover,

$$b_3 b_2 - b_4 b_1 = \frac{k_v I_1 I_2}{D(-S)} (I_2 mgL D + k_p I_1 E^2) > 0,$$

$$b_3 b_2 b_1 - b_4 b_1^2 - b_3^2 b_0 = \frac{k_v^2 I_1 I_2^3 (mgL)^2}{D} > 0.$$

Thus, all fourth-order Routh-Hurwitz conditions hold, and $\nabla_x F(0, \theta)$ is Hurwitz. Since the closed-loop vector field is continuously differentiable, x^* is locally exponentially stable. Therefore, there exists a forward-invariant neighborhood $\mathcal{N}$ of x^* in which exponential convergence holds. ■

2) Controller Parameter Optimization and Simulation Results: We adopt the benchmark example in [17], using the same system parameters. The controller parameters reported therein are used as the initialization for the policy-gradient updates and also serve as the baseline for comparison:

$$I_1 = 0.1, \quad I_2 = 0.2, \quad mgL = 10,$$

$$a_1 = 1, \quad a_2 = -1.5, \quad a_3 = 6, \quad k_p = 3.75, \quad k_v = 10.$$

The projected gradient method is employed to ensure that the updated controller parameters remain within the feasible set defined in (21). For comparison, the state trajectory under the baseline controller from [17] is shown in Figure 3A, while the trajectory obtained using the controller optimized by Algorithm 1 is shown in Figure 3B. The optimized controller drives the system to steady state substantially faster than the baseline controller. The corresponding baseline and optimized control inputs are compared in Figure 3C, where the optimized controller achieves the desired performance with reduced control effort. Finally, the evolution of the controller parameters is shown in Figure 3D, demonstrating smooth convergence of the policy-gradient updates.

Figure 4 shows the convergence of the Gauss-Newton iterations for the inertia-wheel pendulum across different trajectory horizons. The parallel state-trajectory solver converges for all tested horizons, consistent with Theorem 3.2. After several initial iterations, the error decreases rapidly, qualitatively supporting the local quadratic convergence established in Corollary 3.2. Nevertheless, longer horizons may require additional iterations before the iterates enter the local region in which quadratic convergence becomes dominant.

V. CONCLUSIONS

This letter develops a time-parallel policy-optimization framework for discrete-time nonlinear control-affine systems. The policy gradient is expressed in terms of forward state and backward costate trajectories, which are evaluated in parallel using Gauss-Newton iterations and associative scans. We establish that the Gauss-Newton state solver converges locally at a horizon-independent linear rate, while also enjoying global finite-step convergence, with the worst-case number of iterations bounded by the rollout horizon T . Owing to its linear structure, the costate rollout is recovered exactly in a single Gauss-Newton iteration once the state trajectory is fixed. Numerical results for LQR and an inertia-wheel pendulum demonstrate the effectiveness and computational efficiency of the proposed approach. Future work will investigate continuous-time extensions and time-parallel sampling-based control methods.

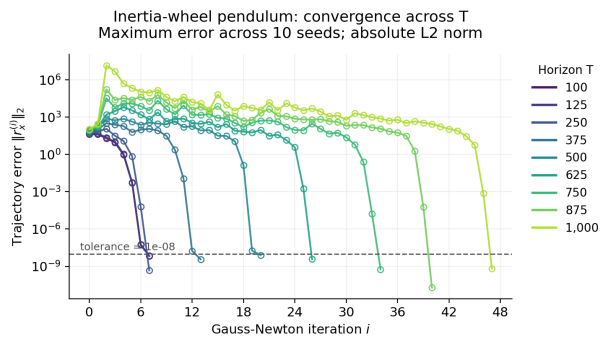


Fig. 4. Gauss-Newton convergence for the inertia-wheel pendulum across nine trajectory horizons ranging from ($T = 100$) to ($T = 1000$). Each curve shows the maximum absolute trajectory error over 10 random seeds, measured against the sequential rollout. All tested horizons reach the (10^{-8}) tolerance (dashed line).

REFERENCES

- [1] H. K. Khalil, *Nonlinear Systems*, 3rd ed. Upper Saddle River, NJ, US: Prentice Hall, 2002.
- [2] M. Krstic, P. V. Kokotovic, and I. Kanellakopoulos, *Nonlinear and Adaptive Control Design*. NJ, US: John Wiley & Sons, Inc., 1995.
- [3] S. Massaroli, M. Poli, F. Califano, J. Park, A. Yamashita, and H. Asama, "Optimal energy shaping via neural approximators," *SIAM Journal on Applied Dynamical Systems*, vol. 21, no. 3, pp. 2126–2147, 2022.
- [4] D. Larby and F. Forni, "Optimal virtual model control for robotics: Design and tuning of passivity-based controllers," *IEEE Transactions on Robotics*, vol. 42, pp. 439–454, 2026.
- [5] B. Hu, K. Zhang, N. Li, M. Mesbahi, M. Fazel, and T. Başar, "Toward a theoretical foundation of policy optimization for learning control policies," *Annual Review of Control, Robotics, and Autonomous Systems*, vol. 6, no. 1, pp. 123–158, 2023.
- [6] M. Fazel, R. Ge, S. Kakade, and M. Mesbahi, "Global convergence of policy gradient methods for the linear quadratic regulator," in *International conference on machine learning*. PMLR, 2018, pp. 1467–1476.
- [7] L. Cui, Z.-P. Jiang, and E. D. Sontag, "Small-disturbance input-to-state stability of perturbed gradient flows: Applications to LQR problem," *Systems & Control Letters*, vol. 188, p. 105804, 2024.
- [8] W. Jin, Z. Wang, Z. Yang, and S. Mou, "Pontryagin differentiable programming: An end-to-end learning and control framework," *Advances in Neural Information Processing Systems*, vol. 33, pp. 7979–7992, 2020.
- [9] Y. H. Lim, Q. Zhu, J. Selfridge, and M. Firmansyah, "Parallelizing non-linear sequential models over the sequence length," in *International Conference on Learning Representations*, vol. 2024, 2024, pp. 55 334–55 360.
- [10] G. E. Blelloch, "Prefix sums and their applications," School of Computer Science, Carnegie Mellon University, Tech. Rep. CMU-CS-90-190, Nov. 1990.
- [11] F. Danieli, M. Sarabia, X. Suau Cuadros, P. Rodriguez, and L. Zappella, "DeepPCR: Parallelizing sequential operations in neural networks," *Advances in Neural Information Processing Systems*, vol. 36, pp. 47 598–47 625, 2023.
- [12] X. Gonzalez, L. Kozachkov, D. Zoltowski, K. Clarkson, and S. Linderman, "Predictability enables parallelization of nonlinear state space models," *Advances in Neural Information Processing Systems*, vol. 38, pp. 19 101–19 147, 2025.
- [13] D. Silver, G. Lever, N. Heess, T. Degris, D. Wierstra, and M. Riedmiller, "Deterministic policy gradient algorithms," in *International conference on machine learning*. Pmlr, 2014, pp. 387–395.
- [14] G. Hewer, "An iterative technique for the computation of the steady state gains for the discrete optimal regulator," *IEEE Transactions on Automatic Control*, vol. 16, no. 4, pp. 382–384, 1971.
- [15] G. E. Blelloch, "Scans as primitive parallel operations," *IEEE Transactions on computers*, vol. 38, no. 11, pp. 1526–1538, 1989.
- [16] J. Bradbury, R. Frostig, P. Hawkins, M. J. Johnson, Y. Katariya, C. Leary, D. Maclaurin, G. Necula, A. Paszke, J. VanderPlas, S. Wanderman-Milne, and Q. Zhang, "JAX: composable transformations of Python+NumPy programs," 2018. [Online]. Available: <http://github.com/jax-ml/jax>
- [17] R. Ortega, M. W. Spong, F. Gómez-Estern, and G. Blankenstein, "Stabilization of a class of underactuated mechanical systems via interconnection and damping assignment," *IEEE transactions on automatic control*, vol. 47, no. 8, pp. 1218–1233, 2002.